Faster exact counting for formulas with explicitly computable low-communication leaf protocols

Valentine Kabanets, Sajin Koroth, Zhenjian Lu
Dimitrios Myrisiotis, Igor C. Oliveira

doi:10.4230/LIPIcs.CCC.2020.15; CC BY 3.0 Unported

Editorial note. Editable transcription of complete formally published CCBY3.0 Unported SAT main proof; raw nativev1 nonexclusive/private. FullSection5.1–5.3 through main theorem proof retained with actual explicitcommunication/time model and requiredSection2 facts. Deterministic/randomized same-framework cases combined, laterAC0 sketch omitted. Original author parameter/index/probability expressions retained without mathematical repair, new assumptions or replacement proof. Five-author metadata unchanged and wrapper split into two lines; correctedSet/ceil/floor handling preserved in independentpreamble. Noinput/include/assets. All production/validationYupeng; root owns compilation.

Allowed standard prerequisites (editorial).

- Formula approximating degree, communication rectangles, formula decomposition and narrow matrix multiplication. FormalSection2 approximation/model facts and fullSection5.1–5.3 source-local explicit approximation/rectangle evaluation algorithm; standard quantum/formula/matrix prior results cited

Theorem 5 (Satisfiability algorithms)**.** *The following results hold.*

1. *There is a deterministic* #SAT *algorithm for* $\mathsf{FORMULA}[s] \circ \mathcal{G}$ *that runs in time*
$$2^{n-t}, \text{ where } t = \Omega\left(\frac{n}{\sqrt{s} \cdot \log^2(s) \cdot D(\mathcal{G})}\right).$$
2. *There is a randomized* #SAT *algorithm for* $\mathsf{FORMULA}[s] \circ \mathcal{G}$ *that runs in time*
$$2^{n-t}, \text{ where } t = \Omega\left(\frac{n}{\sqrt{s} \cdot \log^2(s) \cdot R_{1/3}(\mathcal{G})}\right)^{1/2}.$$

2 Preliminaries

2.1 Notation

Let $n \in \mathbb{N}$; we denote $\{1, \ldots, n\}$ by $[n]$, and denote by U_n the uniform distribution over $\{0,1\}^n$. We use $\widetilde{O}(\cdot)$ (and $\widetilde{\Omega}(\cdot)$) to hide polylogarithmic factors. That is, for any $f\colon \mathbb{N} \to \mathbb{N}$, we have that $\widetilde{O}(f(n)) = O(f(n) \cdot \mathrm{polylog}(f(n)))$.

In this paper, we will mainly use $\{-1,1\}$ as the Boolean basis. In some parts of this paper, we will use the $\{0,1\}$ basis for the simplicity of the presentation. This will be specified in corresponding sections.

2.2 De Morgan formulas and extensions

Definition 8. *An n-variate* de Morgan formula *is a directed rooted tree; its non-leaf vertices (henceforth,* internal gates*) take labels from $\{\mathrm{AND}, \mathrm{OR}, \mathrm{NOT}\} = \{\wedge, \vee, \neg\}$ and its leaves (henceforth,* variable gates*) take labels from the set of variables $\{x_1, \ldots, x_n\}$. Each internal gate has bounded in-degree (henceforth,* fan-in*); the* NOT *gate in particular has fan-in* 1 *and every variable gate has fan-in* 0*. The* size *of a de Morgan formula is the number of its leaf gates.*

In this work, we denote by $\mathsf{FORMULA}[s]$ the class of Boolean functions computable by size-s de Morgan formulas. Let $\mathcal{G}$ denote some class of Boolean functions; then, we denote by $\mathsf{FORMULA}[s] \circ \mathcal{G}$ the class of functions computable by some size-s de Morgan formula where its leaves are labelled by functions in $\mathcal{G}$.

2.3 Approximating polynomials

Definition 9 (Point-wise approximation)**.** *For a Boolean function $f\colon \{-1,1\}^n \to \{-1,1\}$, we say that the function $\tilde{f}\colon \{-1,1\}^n \to \mathbb{R}$ ε-approximates f if for every $z \in \{-1,1\}^n$,*

$$\left| f(z) - \tilde{f}(z) \right| \leq \varepsilon.$$

We will need the following powerful result for the approximating degree of de Morgan formulas.

Theorem 10 ([7], see also [2])**.** *Let $s > 0$ be an integer and $0 < \varepsilon < 1$. Any de Morgan formula $F\colon \{-1,1\}^n \to \{-1,1\}$ of size s has a ε-approximating polynomial of degree $d = O(\sqrt{s} \cdot \log(1/\varepsilon))$. That is, there exists a degree-d polynomial $p\colon \{-1,1\}^n \to \mathbb{R}$ over the reals such that for every $z \in \{-1,1\}^n$,*

$$|p(z) - F(z)| \leq \varepsilon.$$

Note that Theorem 10 still holds if we use $\{0,1\}$ as the Boolean basis.

2.4 Communication complexity

We use standard definitions from communication complexity. In this paper we consider the standard two party model of Yao and its generalizations to multiparty setting. We denote deterministic communication complexity of a Boolean function by $D(f)$ in the two party setting. We refer to [5] for standard definitions from communication complexity.

Definition 11. *Let $f\colon \{0,1\}^n \to \{0,1\}$ be a Boolean function. The* communication matrix of f*, namely M_f, is a $2^{n/2} \times 2^{n/2}$ matrix defined by $(M_f)_{x,y} := f(x,y)$.*

Definition 12. *A* rectangle *is a set of the form $A \times B$, for $A, B \subseteq \{0,1\}^n$. A* monochromatic rectangle *is a rectangle S such that for all pairs $(x,y) \in S$ the value $f(x,y)$ is the same.*

Lemma 13. *Let* Π *be a protocol that computes* $f\colon \{0,1\}^n \to \{0,1\}$ *with at most* D *bits of communication. Then,* Π *induces a partition of* M_f *into at most* 2^D *monochromatic rectangles.*

Given a protocol, its *transcript* is the sequence of bits communicated.

Lemma 14. *For every transcript* z *of some communication protocol, the set of inputs* (x, y) *that generate* z *is a rectangle.*

Below, we recount the definitions of two multiparty communication models used in this work, namely the number-on-forehead and the number-in-hand models.

Definition 15 ("Number-on-forehead" communication model; informal)**.** *In the* k*-party "number-on-forehead" communication model, there are* k *players and* k *strings* $x_1, \ldots, x_k \in \{0,1\}^{n/k}$ *and player* i *gets all the strings except for* x_i*. The players are interested in computing a value* $f(x_1, \ldots, x_k)$*, where* $f\colon \{0,1\}^n \to \{0,1\}$ *is some fixed function. We denote by* $D^{(k)}(f)$ *the number of bits that must be exchanged by the best possible number on forehead protocol solving* f*.*

We also use the following weaker communication model.

Definition 16 ("Number-in-hand" communication model; informal)**.** *In the* k*-party "number-in-hand" communication model, there are* k *players and* k *strings* $x_1, \ldots, x_k \in \{0,1\}^{n/k}$ *and player* i *gets only* x_i*. The players are interested in computing a value* $f(x_1, \ldots, x_k)$*, where* $f\colon \{0,1\}^n \to \{0,1\}$ *is some fixed function. We denote by* $D^{(k\text{-NIH})}(f)$ *the number of bits that must be exchanged by the best possible communication protocol.*

Note that $D^{(k\text{-NIH})}(f) \leq (1 - 1/k) \cdot n + 1$, for any n-variate Boolean function f, as if $k-1$ players write on the blackboard their string, then the player that did not reveal her input may compute $f(x_1, \ldots, x_k)$ on her own and then publish it.

For the communication models mentioned above, there are also bounded-error randomized versions, denoted by R_δ, $R_\delta^{(k)}$, and $R_\delta^{(k\text{-NIH})}$, respectively, where $0 < \delta < 1$ is an upper bound on the error probability of the protocol. In this setting, the players have access to some shared random string, say r, and the aforementioned error probability of the protocol is considered over the possible choices of r. Moreover, we require the error to be at most δ on each fixed choice of inputs.

We can extend the definitions of the communication complexity measures, defined above, to classes of Boolean functions, in a natural way. That is, for any communication complexity measure $M \in \left\{D, D^{(k)}, D^{(k\text{-NIH})}, R_\delta, R_\delta^{(k)}, R_\delta^{(k\text{-NIH})}\right\}$ and for any class of Boolean functions $\mathcal{G}$, we may define

$$M(\mathcal{G}) := \max_{g \in \mathcal{G}} M(g)\,.$$

We note that throughout this paper, we denote by n the number of input bits for the function regardless the communication models. In the k-party communication setting (either NOF or NIH), we assume without loss of generality that n is divisible by k.

5 Satisfiability algorithms

In this section, we will use $\{0,1\}$ as the Boolean basis.

5.1 Computational efficient communication protocols

Definition 40 (Computational efficient communication protocols)**.** *Let $t\colon \mathbb{N}\times\mathbb{N}\to\mathbb{N}$. We say that a two-party communication protocol is* t-efficient *if for each of the parties, given an input x and some previously sent messages $\pi\in\{0,1\}^*$, the next message to send can be computed in time $t(|x|,|\pi|)$ ($\perp$ is being output if there is no next message). We say that such a protocol is* explicit *if $t(|x|,|\pi|)=2^{o(|x|+|\pi|)}$.*

Lemma 41. *Let $f\colon \{0,1\}^n\to 1$ and let Π be a t-efficient communication protocol for f with communication cost at most D. Then the protocol tree of Π can be output in time $O\left(D\cdot t(n/2,D)\cdot 2^n\cdot 2^D\right)$. That is, there exists an algorithm that outputs a list of all (partial and full) transcripts of length at most D and the rectangles associated with each of the transcripts.*

Proof. It suffices to show that, given an input $x\in\{0,1\}^{n/2}$ and a transcript $\ell\in\{0,1\}^{\le D}$, we can decide whether x belongs to the rectangle indexed by ℓ in time $D\cdot t(n/2,D)$. Suppose x is the input for Alice (resp. Bob), and we want to decide whether x belongs to the rectangle indexed by π. We can carry out the communication task by simulating the behavior of Alice (resp. Bob) using the protocol Π and simulating Bob's (resp. Alice's) behavior using the transcript π, and check whether the messages sent by Alice (resp. Bob) is consistent with the transcript π. This takes time at most $D\cdot t(n/2,D)$. To construct the tree, we do the above for every (partial and full) transcript $\pi\in\{0,1\}^{\le D}$ and every input $x\in\{0,1\}^{n/2}$ for Alice (resp. Bob). The total running time is $O\left(D\cdot t(n/2,D)\cdot 2^n\cdot 2^D\right)$. □

For a protocol Π, we denote by Leaves(Π) the set of full transcripts of Π.

Remark. We note that, in the *white-box* context of the satisfiability problem, there is no need to assume a canonical partition of the input variables among the players. For instance, a helpful partition can either be given as part of the input, or computed by the algorithm. As a consequence, in instantiations of Theorem 5 for a particular circuit class $\mathcal{C}$, it is sufficient to be able to convert the input circuit from $\mathcal{C}$ into some device from $\mathsf{FORMULA}\circ\mathcal{G}$ for which protocols of bounded communication cost can be described.

5.2 Explicit approximating polynomials for formulas

From Theorem 10, we know that every size-s formula has a degree-$O(\sqrt{s})$ polynomial that point-wisely approximates it. In our SAT algorithms, we will need to *explicitly construct* such an approximating polynomial given a formula. One way to do this is to use an *efficient* quantum query algorithm for formulas. It is known that a quantum query algorithm for a function f using at most T queries implies an approximating polynomial for f of degree at most $2T$ [1], and by classically simulating such an quantum algorithm, one can show that the approximating polynomial can be obtained in time that is polynomial in the

number of its monomials, in addition to the time for the classical simulation. For our task, we can use the result of Reichardt [6] which showed an *efficient* quantum algorithm for evaluating size-s formulas with $O\left(\sqrt{s}\cdot\log s\right)$ queries[1]. Here, we present an alternate way to construct approximating polynomials for de Morgan formulas which rely only on the *existence* of such polynomials, without requiring an efficient quantum query algorithm. This "black-box" approach was suggested to us by an anonymous reviewer.

We first need the following structural lemma for formulas.

Lemma 42 ([4, 8])**.** *For every integer $s > 0$, there exists an algorithm such that given a size-s de Morgan formula F, runs in* $\mathrm{poly}(s)$ *time and outputs a top formula F' with $O(\sqrt{s})$ leaves and each leaf of F' is a sub-formula with $O(\sqrt{s})$ input leaves.*

Lemma 43. *For any integer $s > 0$ and any $0 < \varepsilon < 1$, there exists an algorithm of running time $s^{O\left(\sqrt{s}\cdot\log(s)\cdot\log(1/\varepsilon)\right)}$ such that given a de Morgan formula F of size s, outputs an ε-approximating polynomial of degree $O(\sqrt{s}\cdot\log(s)\cdot\log(1/\varepsilon))$ for F. That is, the algorithm outputs a multi-linear polynomial (as sum of monomials) over the reals such that for every $x \in \{0,1\}^n$,*

$$|p(x) - F(x)| \leq \varepsilon.$$

Proof. We first note that it suffices to construct a (1/3)-approximating polynomial for F with degree $D = O(\sqrt{s}\cdot\log(s))$. This is because given a (1/3)-approximating polynomial one can obtain explicitly an ε-approximating polynomial of degree $D\cdot O(\log(1/\varepsilon))$, by feeding $O(1/\varepsilon)$ copies of the (1/3)-approximating polynomial to the polynomial computing MAJORITY on $O(1/\varepsilon)$ bits [2] (see also [8, Appendix B]).

We first invoke Theorem 42 on F to obtain a top formula F' with $t = O(\sqrt{s})$ leaves, each of which is a sub-formula of size $O(\sqrt{s})$. We construct a (1/20)-approximating (multi-linear) polynomial P for the top formula F', which has degree $d_1 = O(s^{1/4})$ by Theorem 10. Note that P can be constructed in time $2^{O(\sqrt{s})}$ because F' has at most $O(\sqrt{s})$ leaves. Next, for each of the t sub-formulas, denoted as $F_1, F_2, \ldots, F_t$, we construct a $(1/(20t))$-approximating polynomial. Note that these polynomials have degree $d_1 = O(s^{1/4}\cdot\log(s))$ and can be constructed in time $2^{O(\sqrt{s})}$. Let's denote these t polynomials as $Q_1, Q_2, \ldots, Q_t$. Now for each Q_i ($i \in [t]$), we define

$$q_i(x) = \frac{Q_i(x) + 1/(20t)}{1 + 1/(10t)}.$$

The final approximating polynomial for F is given as

$$p(x) = P\left(q_1(x), q_2(x), \ldots, q_t(x)\right).$$

Note that p has degree $d_1\cdot d_2 = O(\sqrt{s}\cdot\log(s))$ and can be constructed (as sum of monomials) in time $s^{O(\sqrt{s}\cdot\log(s))}$. It remains to show that p (1/3)-approximates F.

[1] It is also known that there exists a quantum query algorithm for evaluating size-s formulas with $O\left(\sqrt{s}\right)$ queries [7], which implies the existence of an approximating polynomial for size-s formulas of degree $O\left(\sqrt{s}\right)$ (see Theorem 10). However, because this algorithm is not known to be efficient, it is unclear whether such an approximating polynomial can be constructed efficiently with respect to the number of monomials.

For $0 \leq q \leq 1$, let N_q be the distribution over $\{0,1\}$ such that $\mathbf{Pr}_{y\sim N_q}[y = 1] = q$. Then for an fixed input $x \in \{0,1\}^s$, we have

$$p(x) = \mathop{\mathbf{E}}_{y_i \sim N_{q_i(x)}} [P(y_1, y_2, \ldots, y_t)]. \tag{1}$$

Let $\mathcal{E}$ be the event that $y_i = F_i(x)$ for all $i \in [t]$. Note that

$$\delta := \mathop{\mathbf{Pr}}_{y_i \sim N_{q_i(x)}} [\neg\mathcal{E}] \leq 1/10. \tag{2}$$

To see Equation (2), note that for every $i \in [t]$, if $F_i(x) = 0$, then $0 \leq q_i(x) \leq 1/(10t)$, which implies

$$\mathop{\mathbf{Pr}}_{y_i \sim N_{q_i(x)}} [y_i \neq F_i(x)] \leq 1/(10t).$$

Similar for the case when $F_i(x) = 1$ (which implies $1 - 1/(10t) < q_i(x) \leq 1$). Then Equation (2) follows from a union bound. Now we can re-write Equation (1) as

$$\begin{aligned} p(x) &= \mathbf{E}[P(y_1, y_2, \ldots, y_t) \mid \mathcal{E}] \cdot \mathbf{Pr}[\mathcal{E}] + \mathbf{E}[P(y_1, y_2, \ldots, y_t) \mid \neg\mathcal{E}] \cdot \mathbf{Pr}[\neg\mathcal{E}] \\ &= (F'(F_1(x), F_2(x), \cdot F_t(x)) \pm 1/20) \cdot (1 - \delta) + \mathbf{E}[P(y_1, y_2, \ldots, y_t) \mid \neg\mathcal{E}] \cdot \delta. \end{aligned}$$

Note that $P(y) \in [-1/(20t), 1+1/(20t)]$ for every $y \in \{0,1\}^t$, and that $\delta \leq 1/10$. A simple calculation shows that

$$p(x) = F'(F_1(x), F_2(x), \ldots, F_t(x)) \pm \frac{1}{3},$$

as desired. □

5.3 The #SAT algorithm

In this subsection, we present our #SAT algorithm.

Theorem 44. *For any integer $s > 0$, there exists a deterministic #SAT algorithm for $\mathsf{FORMULA}[s] \circ \mathcal{G}$, where $\mathcal{G}$ is the class of functions with explicit two-party deterministic protocols of communication cost at most D, that runs in time*

$$2^{n - \frac{n}{\sqrt{s} \cdot \log^2(s) \cdot D}}.$$

In the case $\mathcal{G}$ is the class of functions with explicit randomized protocols of communication cost at most R, there exists an analogous randomized algorithm with a running time

$$2^{n - \left(\frac{n}{\sqrt{s} \cdot \log^2(s) \cdot R}\right)^{1/2}}.$$

The algorithm is based on the framework for designing satisfiability algorithms developed by Williams [9]. The idea is to transform a given circuit into a "sparse polynomial" and solve satisfiability by evaluating the polynomial on all points in a faster-than-brute-force manner.

We first need the following fast matrix multiplication algorithm for "narrow" matrices.

Theorem 45 ([3])**.** *Multiplication of an $N \times N^{.172}$ matrix with an $N^{.172} \times N$ matrix can be done in $O(N^2 \log^2 N)$ arithmetic operations over any field.*

For an even number $n > 0$, and $x \in \{0,1\}^n$, we denote by x^{L} (resp. x^{R}) the first half of x and $x^{\mathrm{R}} \in \{0,1\}^{n/2}$ the second half. We now prove Theorem 44.

Proof of Theorem 44. We first prove the deterministic case.

Let $C = F(g^1, g^2 \ldots, g^s)$ be a device in $\mathsf{FORMULA} \circ \mathcal{G}$ where F is a formula and $g^1, g^2, \ldots, g^s$ are functions that have a explicit communication protocol of cost at most D. The first step is to output the protocol tree for each g^i ($i \in [s]$). Since each g^i has explicit protocol of cost at most D, by Theorem 41, these protocol trees can be output in time $s \cdot 2^{n/2+D+o(n)} \leq 2^{n/1.9}$ (here we assume $D = o(n)$ otherwise the theorem holds trivially).

Let n' be an integer whose value is determined later. Let T be a set of n' variables such that T contains $n'/2$ variables from the first half of the n variables and the rest are from the second half. For a partial assignment $z \in \{0,1\}^{n'}$ to T, denote by C_z the restricted function of C where the variables in T are fixed according to z. To count the number of satisfying assignments of C, we need to compute the following quantity:

$$\sum_{x \in \{0,1\}^{n-n'}} \sum_{z \in \{0,1\}^{n'}} C_z(x). \tag{3}$$

Now consider

$$Q(x) = \sum_{z \in \{0,1\}^{n'}} C_z(x).$$

We will try to obtain the value of $Q(x)$ for every $x \in \{0,1\}^{n-n'}$, in time about $2^{n-n'}$, which will allow us to compute the quantity in Equation (3) in time $O(2^{n-n'})$ by summing $Q(x)$ over all the x's. We do this by first transforming Q into an *approximating* polynomial with not-too-many monomials, and each monomial is a product of *functions that only rely on either the first or the second half of x*. With such a polynomial, we can perform fast multipoint evaluation using the fast matrix multiplication algorithm in Theorem 45.

For each $z \in \{0,1\}^{n'}$, we view the formula C_z as $F(g_z^1, g_z^2, \ldots, g_z^s)$, where F is the de Morgan formula part of C_z and $g_z^1, g_z^2, \ldots, g_z^s$ are the leaf gates. Let us now replace F by a ε-approximating polynomial p, where $\varepsilon = 1/\left(3 \cdot 2^{n'}\right)$, using Theorem 43. Note that the degree of p is at most

$$d \leq O(\sqrt{s} \cdot \log(s) \cdot \log(1/\varepsilon)) \leq O(\sqrt{s} \cdot \log(s) \cdot n').$$

Now consider the following

$$Q'(x) = \sum_{z \in \{0,1\}^{n'}} p(g_z^1(x), g_z^2(x), \ldots, g_z^s(x)).$$

First, note that by the value that we've chosen for the approximating error ε, we have that, for every x,

$$|Q'(x) - Q(x)| \leq 2^{n'} \cdot \varepsilon = 1/3.$$

In other words, given $Q'(x)$, we can recover the value of $Q(x)$, which is supposed to be an integer.

Next, we perform fast multipoint evaluation on Q'. First of all, we re-write Q' as follows:

$$Q'(x) = \sum_{z\in\{0,1\}^{n'}} \sum_{\substack{S\subseteq[s]:\\ |S|\leq d}} \hat{p}(S)\cdot\prod_{i\in S} g_z^i(x). \tag{4}$$

Now let Π_i be the protocol of g^i, we can re-write g_z^i as follows:

$$g_z^i(x) = \sum_{\pi_i\in\text{Leaves}(\Pi_i)} \alpha^i\left(z^{\text{L}}x^{\text{L}},\pi_i\right)\cdot\beta^i\left(z^{\text{R}}x^{\text{R}},\pi_i\right), \tag{5}$$

where $\alpha^i\left(z^{\text{L}}x^{\text{L}},\pi_i\right)$ (resp. $\beta^i\left(z^{\text{R}}x^{\text{R}},\pi_i\right)$) is 1 if and only if $\left(z^{\text{L}}x^{\text{L}}\right)$ (resp. $\left(z^{\text{R}}x^{\text{R}}\right)$) belongs to the rectangle indexed by π_i and the function value of that rectangle is 1. Note that for each $i\in[s]$, given the pre-computed protocol tree of the Π_i, α^i and β^i can be computed in polynomial time (for example, using binary search). After plugging Equation (5) into Equation (4) for every $i\in[s]$ and rearranging, we get

$$Q'(x) = \sum_{z\in\{0,1\}^{n'}} \sum_{\substack{S\subseteq[s]:\\ |S|\leq d}} \sum_{\substack{\vec{\pi}=(\pi_i)_{i\in S}:\\ \pi_i\in\text{Leaves}(\Pi_i)}} \hat{p}(S)\cdot\prod_{i\in S}\alpha^i\left(z^{\text{L}}x^{\text{L}},\pi_i\right)\cdot\prod_{i\in S}\beta^i\left(z^{\text{R}}x^{\text{R}},\pi_i\right). \tag{6}$$

Note that Q' can be expressed as the sum of at most m terms, where

$$m \leq 2^{n'}\cdot s^{O(\sqrt{s}\cdot\log(s)\cdot n')}\cdot 2^{O(\sqrt{s}\cdot\log(s)\cdot n'\cdot D)} \leq 2^{O(\sqrt{s}\cdot\log^2(s)\cdot D\cdot n')}.$$

Note that given Theorem 43, we can obtain Q' in time

$$2^{O(\sqrt{s}\cdot\log^2(s)\cdot D\cdot n')}. \tag{7}$$

Next, we construct a $2^{(n-n')/2}\times m$ matrix A and a $m\times 2^{(n-n')/2}$ matrix B as follows:

$$A_{x^{\text{L}},(z,S,\vec{\pi})} = \hat{p}(S)\cdot\prod_{i\in S}\alpha^i\left(z^{\text{L}}x^{\text{L}},\pi_i\right),$$

and

$$B_{(z,S,\vec{\pi}),x^{\text{R}}} = \prod_{i\in S}\beta^i\left(z^{\text{R}}x^{\text{R}},\pi_i\right).$$

It is easy to see that for each $x\in\{0,1\}^{n-n'}$,

$$Q'(x) = (A\cdot B)_{x^{\text{L}},x^{\text{R}}}.$$

We now want to compute $A\cdot B$. Therefore, we want $m\leq 2^{\cdot 172(n-n')/2}$ so that computing $A\cdot B$ can be done in time $\tilde{O}(2^{n-n'})$ using Theorem 45. For this we can set n' to be

$$n' = \frac{n}{c\cdot\sqrt{s}\cdot\log^2(s)\cdot D},$$

where $c>0$ is some sufficiently large constant. Together with the running time in Equation (7), The total running time of the algorithm is therefore

$$2^{n-\frac{n}{\sqrt{s}\cdot\log^2(s)\cdot D}}.$$

For the randomized case, for each g^i ($i \in [s]$), we consider a randomized protocol Π_i that has error $\varepsilon' \leq 1/(3 \cdot s \cdot 2^{n'})$, and replace g^i with a randomly picked protocol from Π_i, so we can say that for every $x \in n - n'$, the algorithm computes $Q(x)$ (or $Q'(x)$) with probability at least $2/3$ (via a union bound over all the g^i's and a union bound over all the z's in $\{0,1\}^{n'}$). Then we can repeat the above algorithm poly(n) times and obtain $Q(x)$ for all $x \in \{0,1\}^{n-n'}$ correctly with high probability. Note that the error of any randomized protocol with communication complexity R can be reduced to ε' by blowing up the communication complexity by a factor of $O(\log(1/\varepsilon'))$. In this case the, (as we are considering longer transcripts) the number of terms in Q' (as in Equation (6)) will be

$$2^{O(\sqrt{s} \cdot \log^2(s) \cdot R \cdot (n')^2)},$$

and we need to set accordingly

$$n' = \Omega\left(\frac{n}{\sqrt{s} \cdot \log^2(s) \cdot R}\right)^{1/2},$$

which gives the claimed running time for the randomized case. □

References

[1] Robert Beals, Harry Buhrman, Richard Cleve, Michele Mosca, and Ronald de Wolf. Quantum lower bounds by polynomials. *J. ACM*, 48(4):778–797, 2001. doi.org/10.1145/502090.502097.

[2] Harry Buhrman, Ilan Newman, Hein Röhrig, and Ronald de Wolf. Robust polynomials and quantum algorithms. *Theory Comput. Syst.*, 40(4):379–395, 2007. doi.org/10.1007/s00224-006-1313-z.

[3] Don Coppersmith. Rapid multiplication of rectangular matrices. *SIAM J. Comput.*, 11(3):467–471, 1982. doi.org/10.1137/0211037.

[4] Russell Impagliazzo, Raghu Meka, and David Zuckerman. Pseudorandomness from shrinkage. In *FOCS*, pages 111–119, 2012. ECCC:TR12-057.

[5] Eyal Kushilevitz and Noam Nisan. *Communication Complexity*. Cambridge University Press, 1997.

[6] Ben Reichardt. Faster quantum algorithm for evaluating game trees. In *SODA*, pages 546–559, 2011. arXiv:0907.1623.

[7] Ben Reichardt. Reflections for quantum query algorithms. In *SODA*, pages 560–569, 2011. arXiv:1005.1601.

[8] Avishay Tal. Shrinkage of de Morgan formulae by spectral techniques. In *FOCS*, pages 551–560, 2014. ECCC: TR14-048.

[9] Ryan Williams. Nonuniform ACC circuit lower bounds. *J. ACM*, 61(1):2:1–2:32, 2014. doi.org/10.1109/10.1145/2559903.